Universität Bremen

Advanced Digital Competence
Python Programming: Fourier Analysis

Prepared
By:

Anokwu Clifford Chikezie

MSc. Marine Geosciences (3215406)

Tutor: Dr. Karim Norouzi Moghanjoghi

20/04/2026

1. Introduction

The Fourier Series Formula

Any periodic function can be written as:

$f(x) = a_o + \Sigma A_n \sin(\omega_n x) + \Sigma B_n \cos(\omega_n x)$

- **a_o — (average value)**

This is simply the average (mean) value of your function over one full period.

- **n — the mode number (1, 2, 3, ...)**

Each value of n corresponds to one particular frequency of oscillation. n = 1 is the slowest (fundamental frequency), n = 2 oscillates twice as fast, n = 3 three times as fast, and so on. In the code, N is the total number of modes.

- **ω_n — the angular frequency**

$\omega_n = 2\pi n / L$

This controls how fast mode n oscillates. Higher n means higher ω_n and faster oscillation.

- **A_n — the sine coefficient for mode n**

This is a number that tells you how much of the n-th sine wave is present in f(x).

- **B_n — the cosine coefficient for mode n**

Same idea as A_n but for cosine waves.

The formulas for computing these coefficients use integration (finding the area under a curve).

<u>The three formulas are:</u>

$a_0 = (1/L) \int_0^L f(x)\, dx$

$A_n = (2/L) \int_0^L f(x) \sin(\omega_n x)\, dx$

$B_n = (2/L) \int_0^L f(x) \cos(\omega_n x)\, dx$

Numerical Integration

An integral $\int f(x)\, dx$ means 'the area under the curve of f(x)'. When we can't compute it exactly, we approximate it by splitting the area into small slices and summing them up.

Simpson's Rule (used in Part 1)

Instead of straight-line approximations, Simpson's rule fits a parabola through every three consecutive points. This gives a much better approximation for smooth functions using the same number of points. The code uses scipy.integrate.simpson () which implements this automatically.

Discrete Sum (used in Part 2)

When you only have data points (not a formula), the integral becomes a simple sum, and the formulas change to:

$$a_0 \approx (1/L)\, \Sigma_k f_k \Delta x$$

$$A_n \approx (2/L)\, \Sigma_k f_k \sin(\omega_n x_k)\, \Delta x$$

$$B_n \approx (2/L)\, \Sigma_k f_k \cos(\omega_n x_k)\, \Delta x$$

Imports at the Top

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.integrate import simpson
```

These three lines load external Python libraries:

- numpy as np — NumPy is the core numerical library. It lets you work with arrays (lists of numbers) very efficiently and provides functions like np.linspace, np.sin, np.cos, np.zeros, etc. The 'as np' part means you type np.sin instead of numpy.sin as a shortcut.

- matplotlib.pyplot as plt — Matplotlib is the plotting library. plt.plot(), plt.savefig(), plt.show(), all the graphing commands come from here.

- from scipy.integrate import simpson — SciPy is a scientific computing library. This imports specifically the simpson function, which computes the numerical integral of data using Simpson's rule.

2.0 Part 1 — Fourier Coefficients for an Analytical Function

2.1 The Function: compute_fourier_analytical()

This function takes a mathematical formula, evaluates it at many points, and computes the Fourier coefficients using Simpson's rule.

Function signature

```
def compute_fourier_analytical (f, L, N, num_points=1000):
```

The parameters are:

- f — a Python function (callable), and it can be evaluated at any x value.
- L — the period length. For the test functions, $L = 2\pi \approx 6.283$.
- N — how many Fourier modes to compute. N=5 means you compute $A_1 \ldots A_5$ and $B_1 \ldots B_5$.
- num_points=1000 — how many points to use for numerical integration (default 1000; more = more accurate).

Ensuring even number of intervals for Simpson's rule

```
if num_points % 2 != 0:
    num_points += 1
```

Simpson's rule requires an even number of intervals. The % operator is 'modulo' (remainder after division). If num_points is odd (remainder 1 when divided by 2), we add 1 to make it even. This is a safety check.

Creating the x-axis points

```
x = np.linspace(0, L, num_points + 1)
```

np.linspace(start, stop, count) creates an array of 'count' evenly spaced numbers from start to stop inclusive. So x goes from 0 to L in num_points+1 steps.

Evaluating the function at all x points

fx = f(x)

This calls the function f with the entire array x as input. This evaluates f at every single point simultaneously and stores all results in the array fx. So fx[0] = f(x[0]), fx[1] = f(x[1]), etc.

2.2 Application of Simpson Rule to Compute the Coefficients

Computing a_0 — the average

a0 = (1 / L) * simpson(fx, x=x)

This is the numerical approximation of $a_0 = (1/L) \int_0^L f(x)\, dx$. The simpson() function takes the y-values (fx) and the x-values (x) and computes the area under the curve using Simpson's rule. Dividing by L gives the average value of f over [0, L].

Initialising coefficient arrays

A = np.zeros(N)

B = np.zeros(N)

np.zeros(N) creates an array of N zeros. A will eventually hold [A_1, A_2, ..., A_N] and B will hold [B_1, B_2, ..., B_N]. We start with zeros and fill them in the loop.

The main loop — computing each mode

```
for n in range(1, N + 1):

    omega_n = 2 * np.pi * n / L

    integrand_A = fx * np.sin(omega_n * x)

    integrand_B = fx * np.cos(omega_n * x)


    A[n - 1] = (2 / L) * simpson(integrand_A, x=x)

    B[n - 1] = (2 / L) * simpson(integrand_B, x=x)
```

This loop runs N times — once for each mode n from 1 to N. For each n:

- omega_n = 2 * np.pi * n / L. This calculates $\omega_n = 2\pi n/L$, the angular frequency of mode n.
- integrand_A = fx * np.sin(omega_n * x). This multiplies f(x) by $\sin(\omega_n x)$ at every point simultaneously (NumPy array multiplication). The result is the integrand (the thing being integrated) for A_n.
- integrand_B = fx * np.cos(omega_n * x) . Same but for the cosine coefficient B_n.
- A[n-1] = (2/L) * simpson(...) . Computes the integral of integrand_A using Simpson's rule, multiplies by 2/L, and stores it in position n-1 of the A array (position n-1 because Python arrays are 0-indexed — A[0] holds A_1, A[1] holds A_2, etc.).
- B[n-1] = same thing for cosine.

The index n-1: Python counts from 0. So the first mode (n=1) goes into A[0], second mode (n=2) into A[1], and so on.

Return values

return a0, A, B

The function returns three values: the scalar a_0, and the two arrays A and B of length N each.

2.3 Reconstruct the Fourier

The Function: reconstruct_fourier()

Once you have the coefficients, this function rebuilds the approximated signal.

```
def reconstruct_fourier(x, L, a0, A, B):
    N = len(A)
    result = np.full_like(x, a0, dtype=float)
    for n in range(1, N + 1):
```

```
    omega_n = 2 * np.pi * n / L

    result += A[n-1] * np.sin(omega_n * x) + B[n-1] * np.cos(omega_n * x)

return result
```

- N = len(A) — finds how many modes are in the A array.

- result = np.full_like(x, a0, dtype=float) — creates an array the same shape as x but filled with the value a0. This is the 'starting value' of our reconstruction: $f_n(x)$ starts at a_o everywhere.

- The for loop then adds each mode: for each n, it computes $A_n \sin(\omega_n x) + B_n \cos(\omega_n x)$ and adds it to result. After N iterations, result contains the full truncated Fourier reconstruction $f_n(x)$.

- return result — returns the reconstructed signal as a NumPy array.

2.4 The Test Function: part1_demo()

This function runs the complete demonstration and creates the plots for Part 1.

Setting up the figure

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

fig.suptitle('Part 1 — Fourier Coefficients from Analytical Function', fontsize=14)
```

plt.subplots(2, 2) creates a 2×2 grid of subplots (4 panels). fig is the overall figure, axes is a 2×2 array of individual plot panels. figsize=(14, 10) sets the size in inches. suptitle sets a title for the whole figure.

Test Function 1: f(x) = x

```
L1 = 2 * np.pi

f1 = lambda x: x
```

L1 is the period — here 2π.

f1 is defined using a lambda (anonymous function). lambda x: x means 'a function that takes x and returns x'. This is just f(x) = x — the straight line.

```
for col, N in enumerate([5, 20]):

    a0, A, B = compute_fourier_analytical (f1, L1, N)
```

enumerate([5, 20]) iterates over the list [5, 20], and col takes values 0 and 1 (the column index). So on the first pass col=0, N=5 (5 modes); on the second col=1, N=20 (20 modes). compute_fourier_analytical returns all three coefficient values.

```
x_plot = np.linspace(0, L1, 500)

f_exact = f1(x_plot)

f_approx = reconstruct_fourier(x_plot, L1, a0, A, B)
```

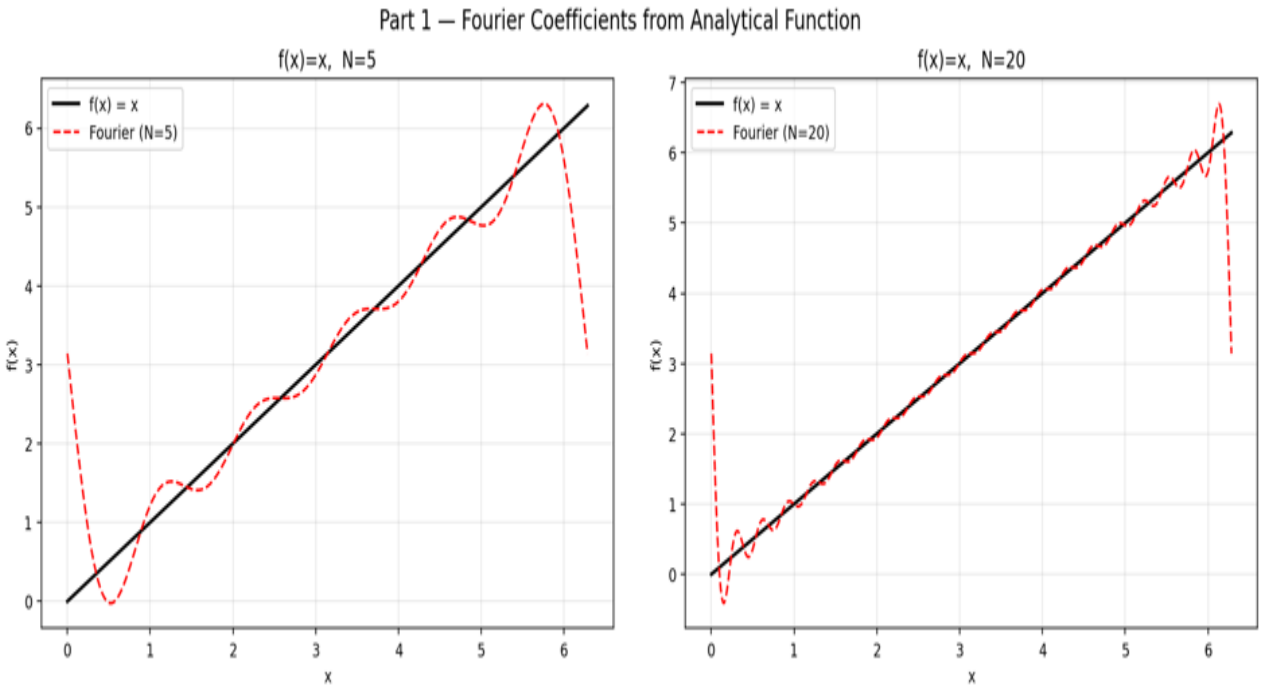
500 points are created for smooth plotting. f_exact is the original function evaluated at those points. f_approx is our Fourier reconstruction. Both are then plotted both to compare visually.

```
ax = axes[0, col]

ax.plot(x_plot, f_exact,  'k-',  lw=2,   label='f(x) = x')

ax.plot(x_plot, f_approx, 'r--', lw=1.5, label=f'Fourier (N={N})')
```

axes[0, col] selects the top row (row 0), col-th column. 'k-' means black solid line. 'r--' means red dashed line. lw is line width. label is the legend text. The f'...' is a Python f-string — it embeds the variable N directly into the string.



When N=5, the reconstruction(dotted red line) looks rough and wiggly and does not perfectly trace the original function (straight black line), but When N=20, it tracks the original function much more closely. This demonstrates Fourier convergence — more modes = better approximation.

3.0 Part 2

3.1 Fourier Coefficient from Discrete Data

In this part raw sampled data points were given, and the task is to compute the Fourier building blocks from the numbers directly.

The formulas change to:

$$a_o \approx (1/L)\, \Sigma_k f_k\, \Delta x$$

$$A_n \approx (2/L)\, \Sigma_k f_k \sin(\omega_n x_k)\, \Delta x$$

$$B_n \approx (2/L)\, \Sigma_k f_k \cos(\omega_n x_k)\, \Delta x$$

where xk = k·Δx (the x-position of the k-th data point).

The Function: compute_fourier_discrete()

def compute_fourier_discrete(f_data, dx, N):

Parameters:

f_data is the array of measured values.

dx is the spacing between consecutive measurements (Δx).

N is how many modes to compute.

Setup:

```
f_data = np.asarray (f_data, dtype=float)
n = len (f_data)
L = n * dx
xk = np.arange(1, n + 1) * dx
```

- np.asarray (..., dtype=float) converts f_data into a NumPy array of floating-point numbers.
- n = len(f_data) — n is the total number of data points.
- L = n * dx — the domain length.

- xk = np.arange(1, n+1) * dx — creates the x-positions of each data point.

3.2 Implementing the discrete formulas for a0, An , and Bn.

Computing a0

```
a0 = (1 / L) * np.sum(f_data) * dx
```

np.sum(f_data) adds up all the data values. Multiplying by dx and dividing by L gives the discrete approximation of the integral a0 = (1/L) ∫ f dx (Compare this to the Simpson's rule version in Part 1 — same concept, simpler implementation).

Creating N zeros of A and B arrays.

```
A = np.zeros(N)
B = np.zeros(N)
```

np.zeros(N) creates an array of N zeros. A will eventually hold [A_1, A_2, ..., A_N] and B will hold [B_1, B_2, ..., B_N]. We start with zeros and fill them in the loop.

The loop for A n and B n

```
for mode in range(1, N + 1):
omega_n = 2 * np.pi * mode / L
A[n-1] = (2/L) * np.sum(f_data * np.sin(omega_n * xk)) * dx
B[n-1] = (2/L) * np.sum(f_data * np.cos(omega_n * xk)) * dx
```

This is identical to Part 1's loop, but instead of simpson() (which needs a formula), it uses np.sum() (which just adds up numbers).

- f_data * np.sin(omega_n * xk) — element-wise multiplication: each f_k is multiplied by $\sin(\omega_n x_k)$.

- np.sum(...) * dx — sums up all products and multiplies by Δx — this is the discrete integral $\Sigma f_k \sin(\omega_n x_k) \Delta x$.

- Multiply by (2/L) to get the final A_n.

3.3 The Demo Function: part2_demo()

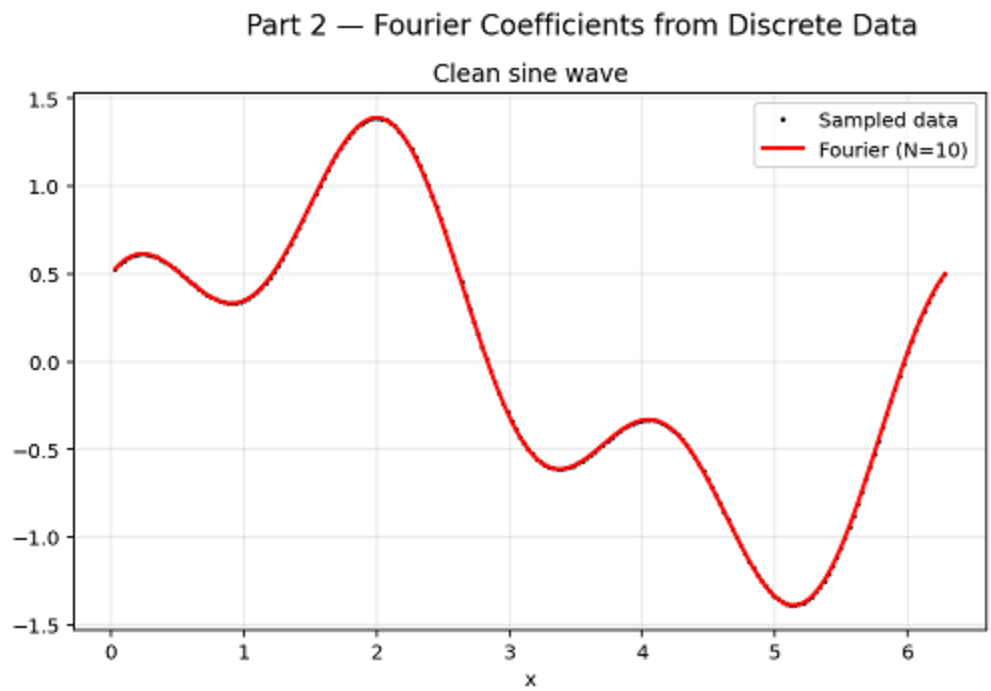
Dataset 1: Clean sampled sine wave

```
n1 = 200
dx1 = 2 * np.pi / n1
xk1 = np.arange(1, n1 + 1) * dx1
N1   = 10 (Number of modes)
f1 = np.sin(xk1) + 0.5 * np.cos(3 * xk1)
```

```
a0, A, B = compute_fourier_discrete(f1, dx1, N1)
L1 = n1 * dx1
f_approx = reconstruct_fourier(xk1, L1, a0, A, B)
```

200 data points are created to sample the function sin(x) + 0.5cos(3x) over [0, 2π]. This is the same function from Part 1, but now treated as if you only have discrete measurements. The coefficients should come out very close to Part 1's results.

```
axes[0].plot(xk1, f1,      'k.',  ms=3,   label='Sampled data')
axes[0].plot(xk1, f_approx, 'r-',  lw=2,   label=f'Fourier (N={N1})')
axes[0].set_title("Clean sine wave")
axes[0].set_xlabel("x")
axes[0].legend()
axes[0].grid(True, alpha=0.3)
```



Part 2 uses the same function sin(x) + 0.5cos(3x) sampled at 200 points. Because the function is smooth and the sampling is fine, the discrete coefficients (red line) traces the sampled data(dotted black line) closely, which indicates a perfect approximation.